Missile Command
Lab 4 - VibeCoding

SRN: PES2UG24CS036

16 October 2026

Assigned repository

https://github.com/SETAPESU26/33_missileCommand
Baseline: b4318a97def23a2fe945a9a78f713c83ebf6990f

Completed work

1. Battery selection filters out destroyed and empty batteries, selects the nearest usable battery, and handles no available battery safely.

2. Explosion colors smoothly change from white through yellow and orange to dark red over their lifetime.

3. City destruction displays a three-second warning. When two or fewer cities survive, the message becomes critical.

4. Every newly crossed 2,000-point milestone rebuilds one destroyed city, with a rebuild message and next-milestone display.

Supporting fixes ensure exact interceptor arrival, prevent waves from clearing flying interceptors, and reject expired blast collisions.

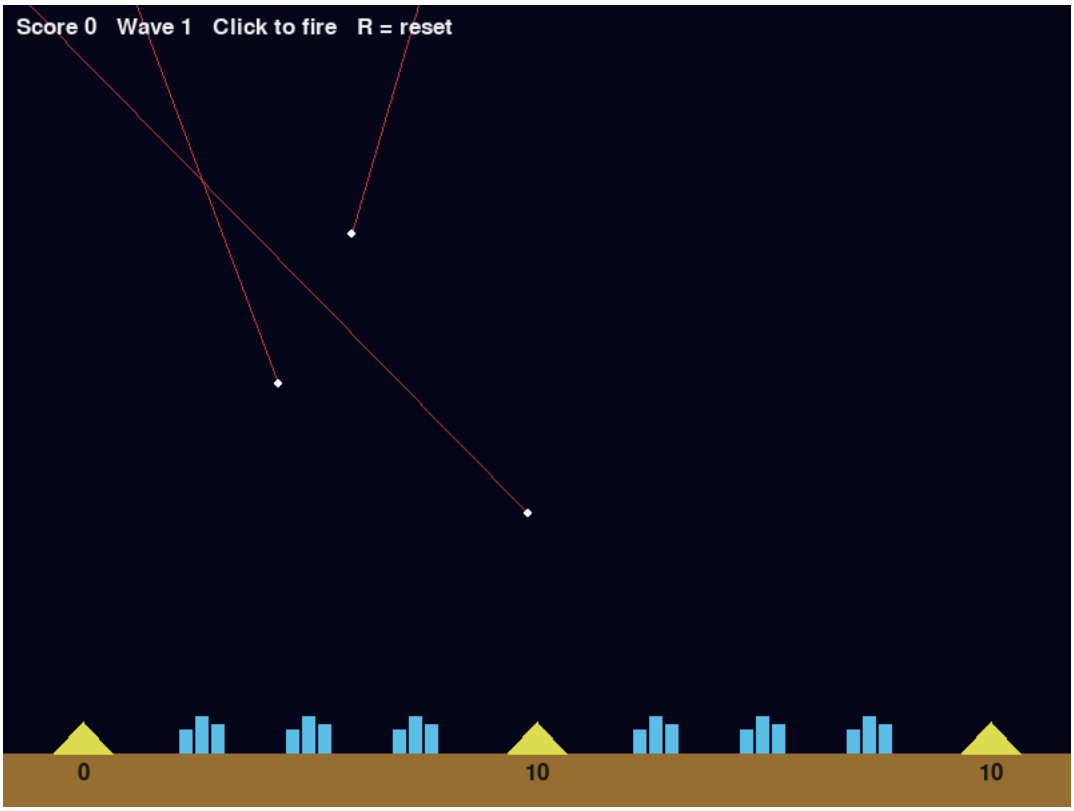
Evidence and verification

26 regression tests passed. Ten unedited Pygame surface screenshots are included. Screenshots replace videos at the student's explicit request.

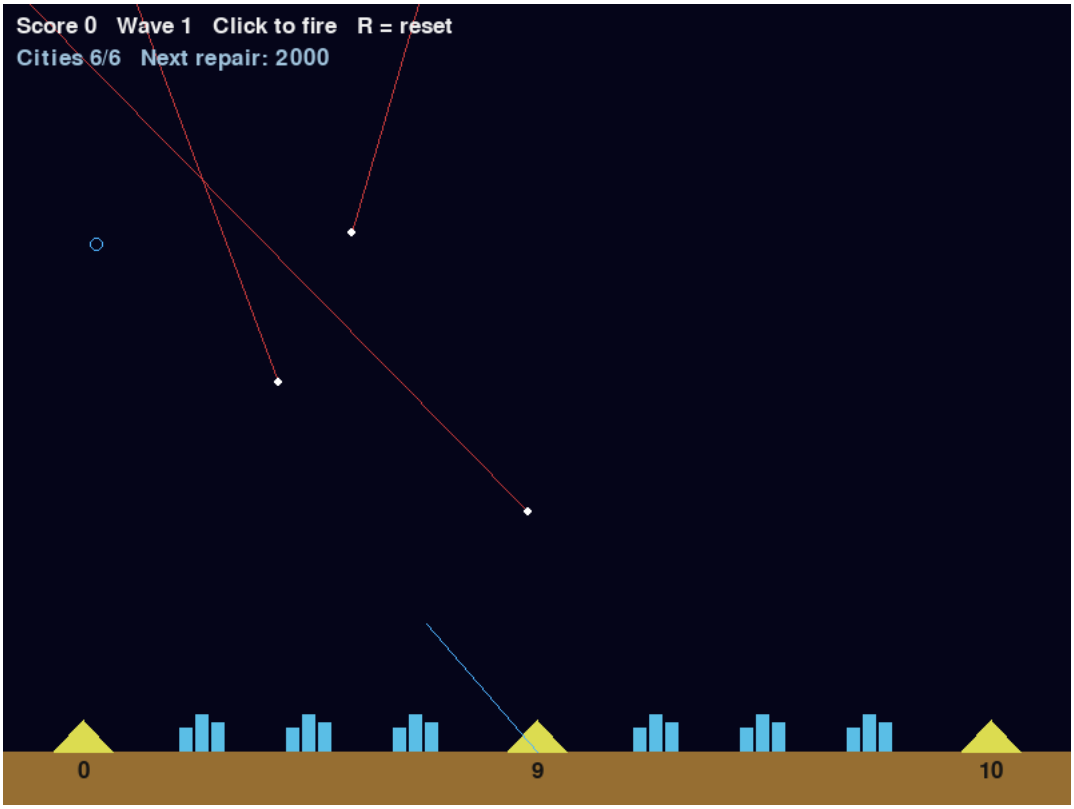
Screenshots use deterministic staged scenarios. Missile positions, explosion ages and the repair starting score are configured by the capture script; the game itself renders every image and performs the launches, impacts and repair collision. These are not an uninterrupted manual gameplay recording.

AI assistance is disclosed in the accompanying actual-message chat export. No separate student prompts have been invented.

Battery selection: before and after

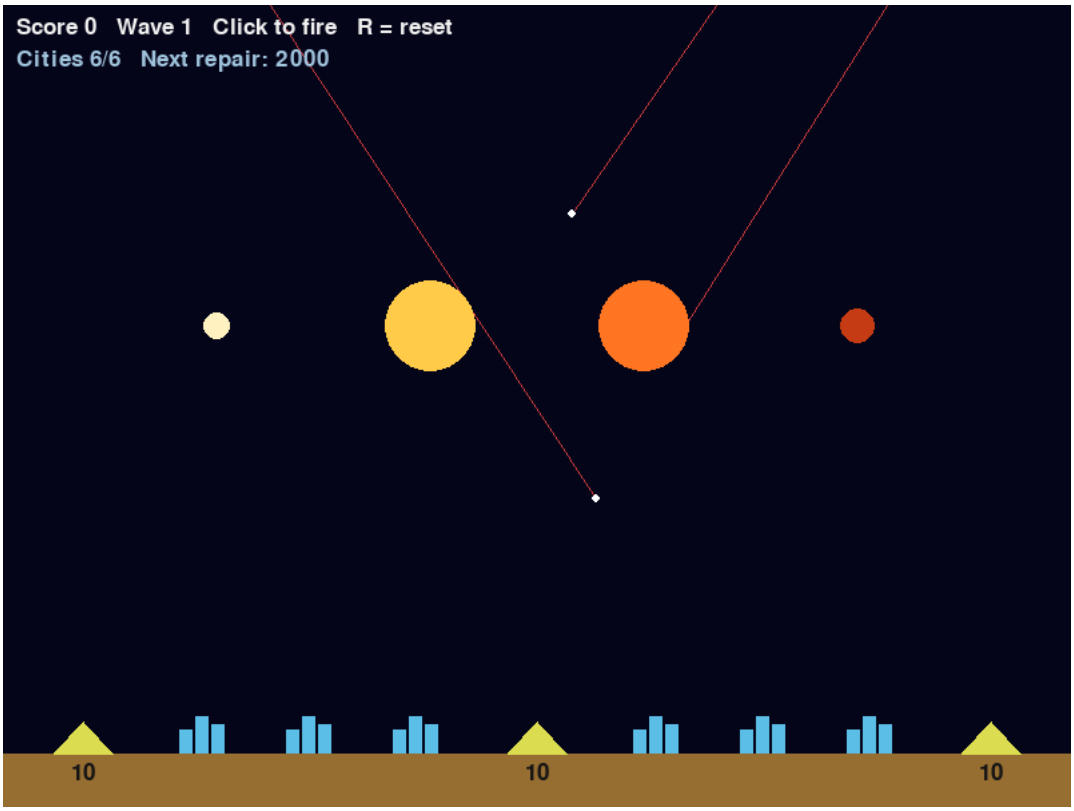


Before: ten launches exhaust the left battery. A further click at (70,180) produces no interceptor; the other batteries retain 10 rounds each.



After: the same click launches from the middle battery. Its ammunition falls to 9; the cyan interceptor trail and target ring are visible.

Explosion colors and destruction warning



Four explosions at progress 0.12, 0.38, 0.62 and 0.85 show the white/yellow-to-red lifetime gradient. These are staged explosion ages.

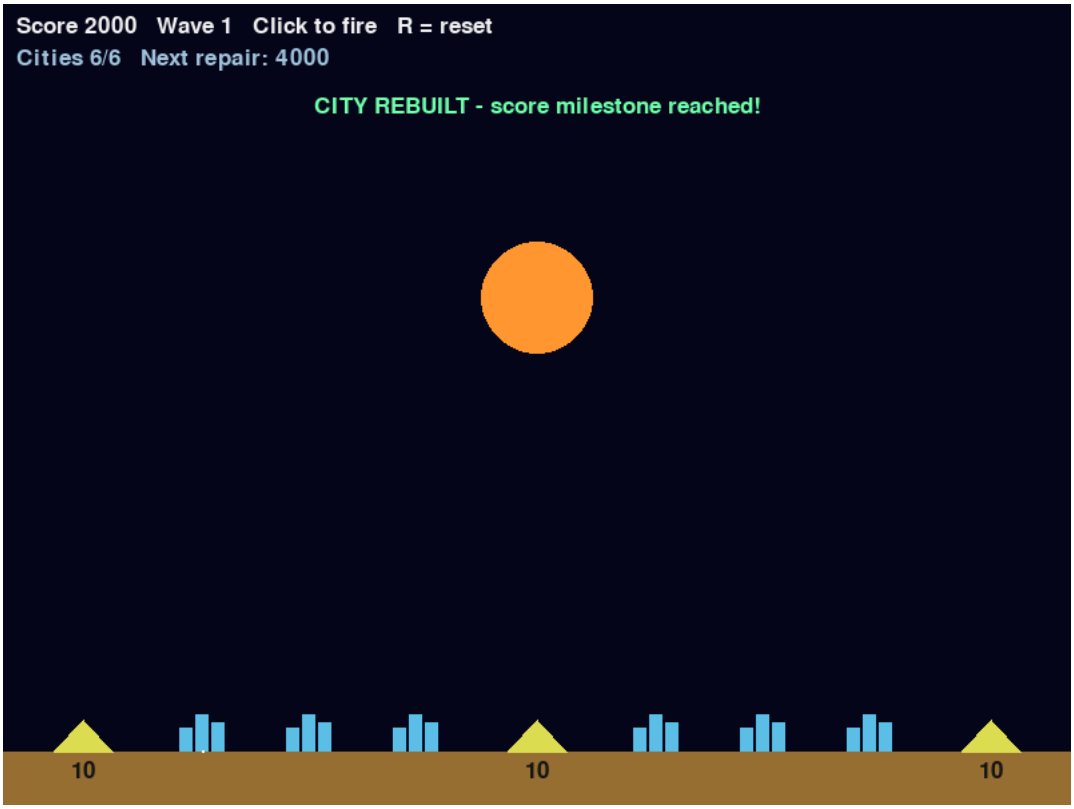


Four actual impact() calls destroy four cities. The HUD shows Cities 2/6 and the critical warning appears for three seconds.

Score-based city repair



Before repair: one city is destroyed and the scenario starts at 1,975 points. The city count is 5/6 and the next threshold is 2,000.



An explosion intercepts a missile for 25 points. At 2,000 points the destroyed city is rebuilt, the count becomes 6/6 and the next threshold is 4,000.

Game over and reset



All six cities are destroyed by impacts. The game enters the lose state and displays ALL CITIES LOST - Press R.



Reset returns to score 0, wave 1, all six cities, three full batteries and the first repair milestone. R-key event dispatch is also tested.

Validation and reproduction

Environment: Python 3.13.15; Pygame 2.6.1; SDL dummy video/audio drivers for automated verification.

Run the game

```
cd Lab-4
python -m pip install -r requirements.txt
python game.py
```

Click above the ground to fire; R resets; close the window to quit.

Run checks and recapture screenshots

```
python -m unittest discover -s tests -v
python tools/capture_screenshots.py --phase after
```

Checks completed

Nearest available battery, depleted/destroyed fallback, mixed unavailability and safe no-ammo behavior.

Exact interceptor arrival, short remaining distances, zero-distance targets and target-centered detonation.

Valid interpolated colors; destruction hook runs once per city; battery impacts skip the hook; alerts expire.

Repair threshold, no repeated award, multiple crossed milestones, no banked repairs and disabled repairs.

Explosion radius/boundary collision, overlapping blast scoring and expired explosion cleanup.

Missile impact, valid spawn targets, wave completion, ammunition replenishment, loss and full reset.

Main-loop dispatch of mouse click, R key and quit events.

Result: 26 tests passed; exit code 0.

Full output: verification.txt. Screenshot state records: screenshots/before_scenarios.json and screenshots/after_scenarios.json.

Submission location

https://github.com/Audatic07/SE_Lab/tree/main/Lab-4
The original README is preserved as UPSTREAM_README.md. All changes are in the personal lab repository; no upstream pull request is required.